

Студент: Сабаев Павел Андреевич, ЕТ - 342

Руководитель: Фомина Анна Алексеевна

Тема: Разработка web-приложения для ведения учета инвестиционных сделок

Архитектура системы

1. Общее описание архитектуры системы

Разрабатываемая система представляет собой серверное веб-приложение для учета инвестиционных сделок пользователей. Система реализована в виде REST API и построена на основе клиент-серверной архитектуры.

В качестве серверной части используется фреймворк FastAPI, обеспечивающий обработку HTTP-запросов и взаимодействие с базой данных. Для хранения данных применяется реляционная база данных SQLite.

Архитектура системы построена по принципу разделения на слои:

- 1) слой представления (API);
- 2) слой бизнес-логики (сервисы);
- 3) слой доступа к данным (ORM SQLAlchemy);
- 4) слой моделей данных (модели и схемы).

Система поддерживает регистрацию и авторизацию пользователей, а также выполнение операций с инвестиционными сделками.

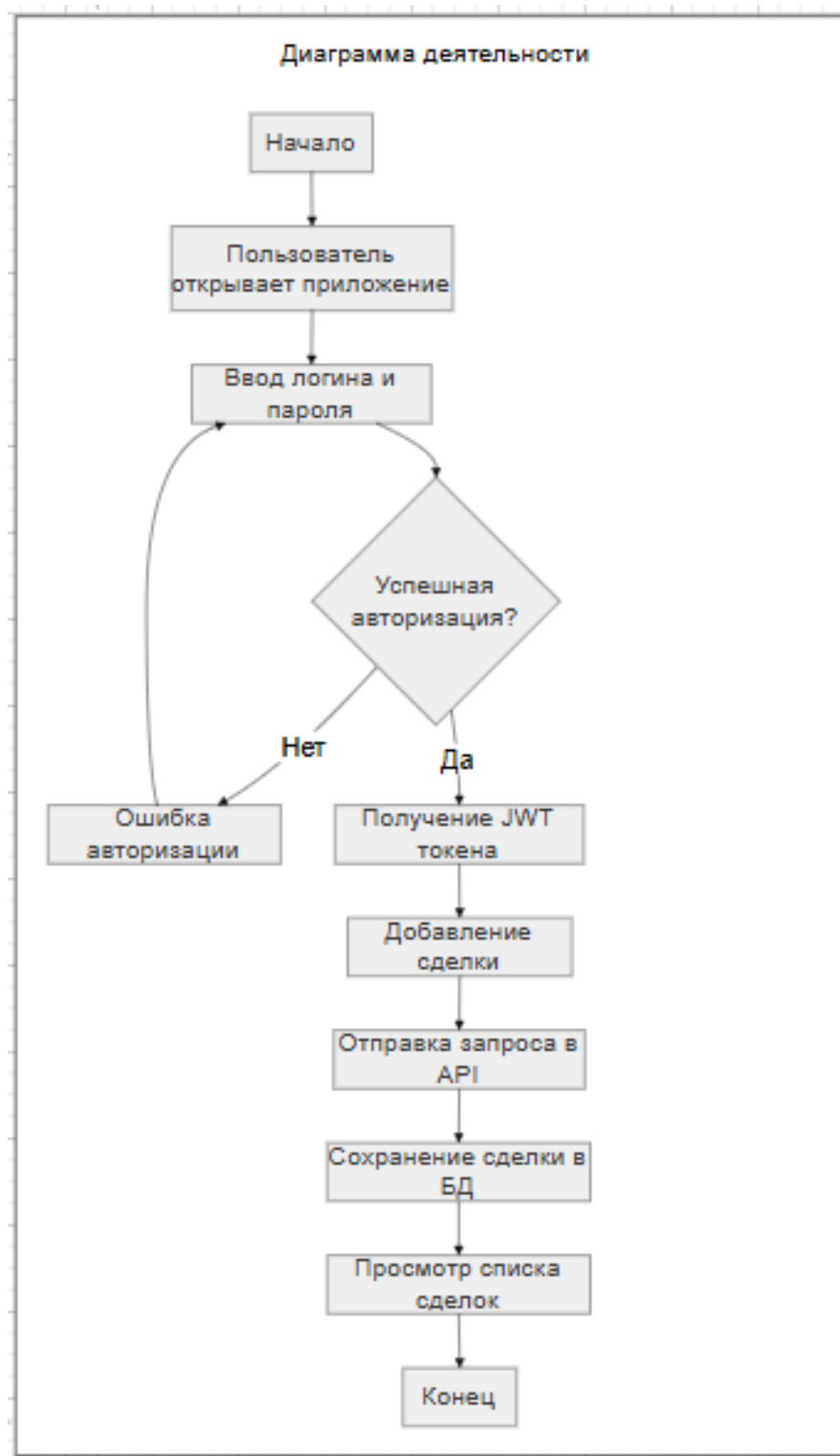


Рисунок 1 – Диаграмма деятельности

1. В данной диаграмме деятельности представлен общий сценарий взаимодействия пользователя с системой.
2. Процесс начинается с открытия приложения пользователем и ввода учетных данных. Далее система проверяет корректность введенных данных. В

случае ошибки пользователь возвращается к этапу ввода, в противном случае происходит успешная авторизация и выдача JWT-токена.

3. После авторизации пользователь получает возможность выполнять операции со сделками, например, добавлять новые сделки. Запрос на добавление отправляется в систему, где данные обрабатываются и сохраняются в базе данных.

4. После этого пользователь может просматривать список своих сделок. Процесс завершается завершением работы пользователя с системой.

5. Диаграмма отражает общий поток действий пользователя и основные этапы взаимодействия с системой.

2. Описание компонентов, составляющих систему

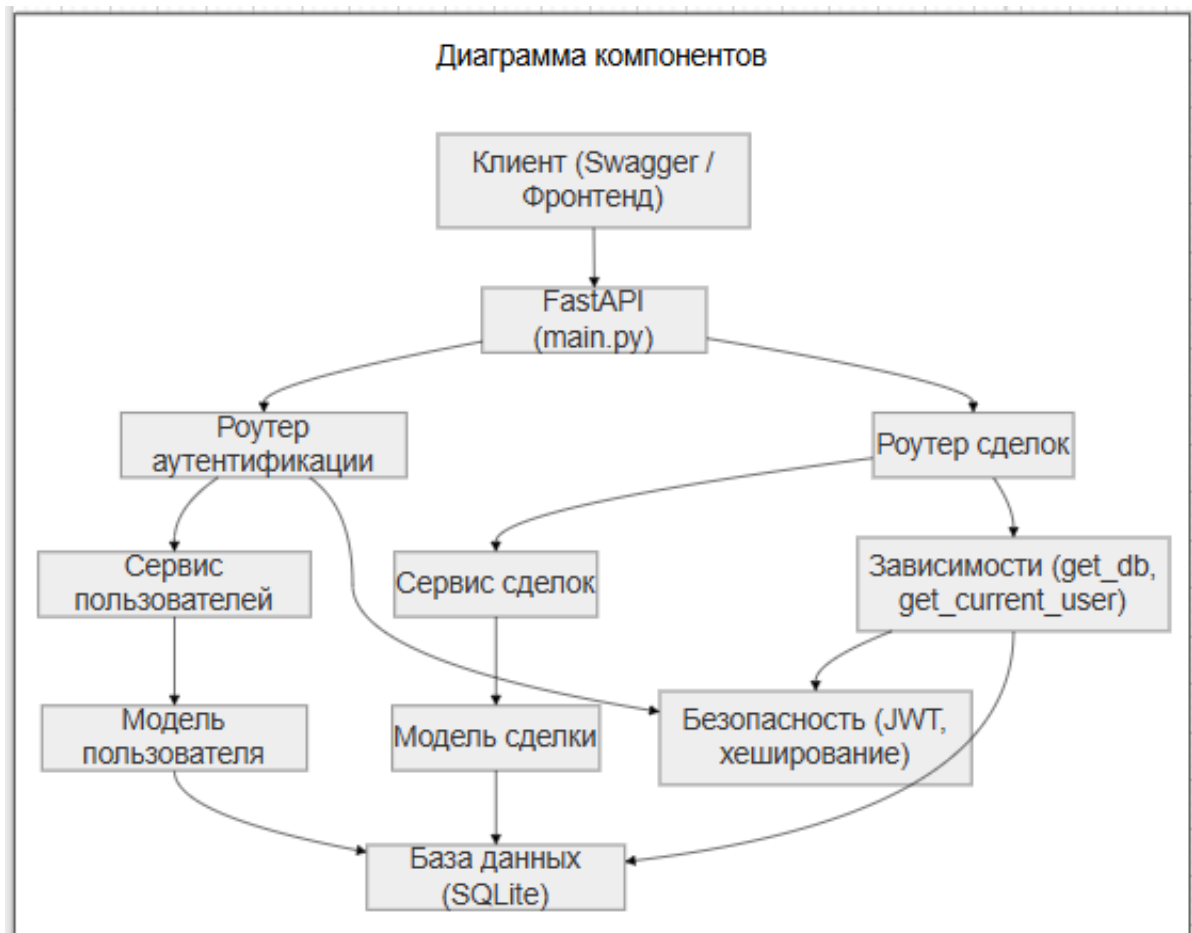


Рисунок 2 – Диаграмма компонентов

1. В данной диаграмме компонентов представлена общая архитектура программной системы и взаимодействие её основных частей.

2. Пользователь (клиент) взаимодействует с системой через интерфейс (Swagger или фронтенд), отправляя HTTP-запросы в приложение, реализованное на базе FastAPI. Приложение выступает в роли центральной точки обработки запросов и распределяет их по соответствующим маршрутам (роутерам).

3. Роутер аутентификации отвечает за регистрацию и авторизацию пользователей, а роутер сделок – за операции, связанные с управлением сделками (создание, получение, обновление, удаление).

4. Каждый роутер использует соответствующий сервисный слой. Сервисы реализуют бизнес-логику приложения: обработку данных, подготовку объектов и взаимодействие с моделями.

5. Модели представляют структуру таблиц базы данных и используются для работы с ORM (SQLAlchemy). Все данные сохраняются и извлекаются из базы данных SQLite.

6. Компонент безопасности отвечает за генерацию и проверку JWT-токенов, а также за хеширование паролей пользователей. Компоненты зависимостей обеспечивают доступ к базе данных и текущему пользователю.

3. Модель базы данных

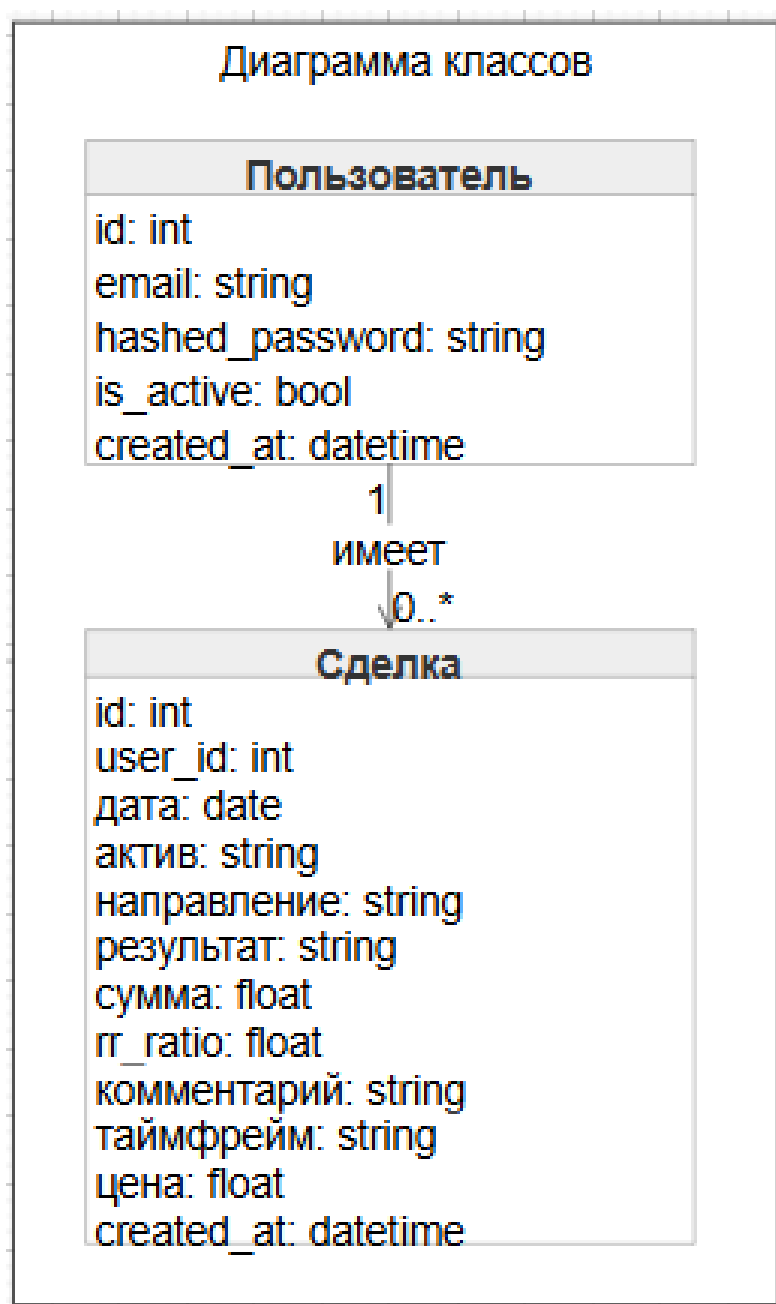


Рисунок 3 – Диаграмма классов

1. В данной диаграмме классов представлена структура данных системы и взаимосвязи между основными сущностями.

2. Основными сущностями являются "Пользователь" и "Сделка". Класс "Пользователь" содержит информацию об учетной записи: идентификатор, электронную почту, зашифрованный пароль, статус активности и дату создания.

3. Класс "Сделка" содержит информацию о торговых операциях пользователя, включая дату сделки, актив, направление (покупка или продажа), результат, сумму, коэффициент риск/прибыль, комментарий, таймфрейм, цену и дату создания записи.

4. Между классами установлена связь "один ко многим": один пользователь может иметь множество сделок, при этом каждая сделка принадлежит только одному пользователю. Связь реализуется через внешний ключ user_id в таблице сделок.

5. Диаграмма отражает логическую структуру базы данных и используется для понимания того, как хранятся и связаны данные в системе.

4. Реализация архитектуры системы

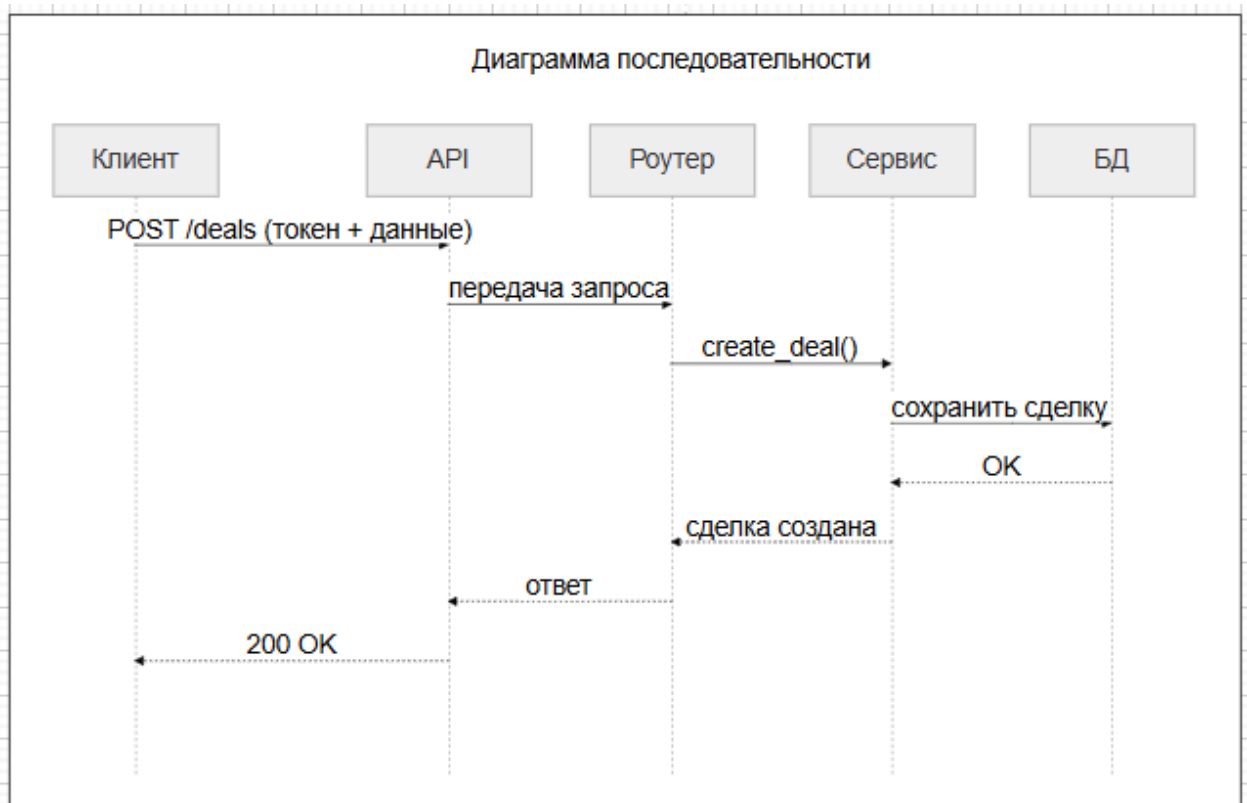


Рисунок 4 – Диаграмма последовательности

1. В данной диаграмме последовательности показан процесс взаимодействия компонентов системы при добавлении новой сделки.

2. Пользователь отправляет запрос на создание сделки, передавая данные сделки и JWT-токен для аутентификации. Запрос поступает в API-приложение, которое перенаправляет его в соответствующий роутер.

3. Роутер обрабатывает запрос и вызывает сервисный слой, передавая данные сделки и идентификатор текущего пользователя. Сервис формирует объект сделки и инициирует его сохранение в базе данных.

4. База данных подтверждает успешное сохранение, после чего результат возвращается обратно по цепочке: от сервиса к роутеру, затем к API и, в конечном итоге, к пользователю.

5. Диаграмма наглядно демонстрирует последовательность вызовов и передачу данных между компонентами системы при выполнении операции.